CS 490 - P2 Report

GrubFindr

Date: March 13, 2026

Team:
Abyan Haq - Project Manager - abyan.haq@uwaterloo.ca
Andre Slavescu - System Architect - aslavesc@uwaterloo.ca
Bilal Nasar - Communication Officer - bnasar@uwaterloo.ca
Jeffrey Qingjie Xuzhang - Information Officer - jqxuzhang@uwaterloo.ca

Introduction

Modern meal hunting has become more difficult than ever, due to a frustrating paradox: on one hand, people now have have access to more options and details than ever before, but on the other hand, it has now become harder than ever to choose what to eat. The majority of diners now find themselves trapped in a cycle of 'analysis, paralysis,' going through google maps, yelp, tiktok and every other app that recommends food in any capacity, yet still not being able to make a decision. This decision, which should have been simple and pleasant, is now becoming a task that people would rather avoid, as there is tons of fragmented data, making it seem impossible to get a straightforward and reliable answer.

GrubFindr came into being to clarify this confusion in the digital world. Instead of making the market even more complicated by launching yet another platform, the solution is like a clever filter which brings rationale to the discovery process once again. Thanks to the real-time data consolidation into a single, easy-to-use interface, GrubFindr makes the search a less troublesome task so that users don't have to spend so much time scrolling and can enjoy their meal at a restaurant. Users enter their location, budget, dietary requirements, and nutritional goals, which is then used by Large Language Models and Vision Language Models to provide a single, context based dish that users will undoubtedly enjoy. This report presents the architectural design of GrubFindr, covering the software architecture, data model, business model, and market analysis.

Competitive Analysis

The food discovery and recommendation space is dominated by general purpose platforms that treat restaurant discovery as a secondary feature. GrubFindr competes primarily with three other companies / offerings: Google Maps, Yelp, and Beli. Each serves a different decision making criteria, but none provide insights on the product / dish such as nutrition-aware personalization that GrubFindr offers.

Google Maps

Google Maps dominates restaurant discovery with over 2 billion monthly active users and 67-70% global market share in mapping. Approximately 62% of consumers discover restaurants through Google. Features include cuisine/price/dietary filters, Gemini-powered review summaries, a "Your Match" personalization score, and integrated reservations via OpenTable. Revenue comes from cost-per-click local search ads (~$2.05 CPC). However, as a general navigation platform, its food personalization is shallow, based on location and search history rather than deep preference modeling. Its algorithms also amplify already-popular restaurants while underrepresenting lesser-known ones.

Yelp

Yelp holds approximately 6% of the online review market with 76 million monthly users and 308 million+ cumulative reviews. Its strengths include comprehensive filtering, integrated waitlist/reservations, and recent AI investments such as Menu Vision (AR menu scanner) and Popular Offerings (LLM-powered dish highlights). Revenue comes from CPC advertising ($444 million in restaurant ad revenue in 2025, down 6% YoY). Despite these AI features, recommendations remain popularity- and location-driven with no deep taste profile learning. The user base skews older (41% aged 55+), leaving younger demographics underserved.

Beli

Beli targets Gen Z with 75 million+ reviews across 30,000 cities, overtaking Yelp in annual review volume in 2024. Its differentiator is a pairwise comparison ranking system (1-10 scale) and social features including friend feeds and taste similarity scores. The app is free, monetizing through an invite-only Supper Club and B2B data insights. Weaknesses include high onboarding friction (must invite 4 friends), geographic bias toward major U.S. cities, and no dietary or nutrition awareness. Approximately 80% of users are under 35.

Market Share Comparison

Estimated market presence in the food discovery and restaurant recommendation space:

Platform	Estimated Monthly Users	Market Position
Google Maps	over 2 billion; approx. 62% of restaurant discovery	Dominant, general purpose platform
Yelp	76 million; approx. 6% of online review market	#2 U.S. review platform
Beli	Undisclosed DAU; over 75M reviews in 30K cities	Fast-growing Gen Z niche
GrubFindr	Pre-launch	Targeting underserved personalization niche

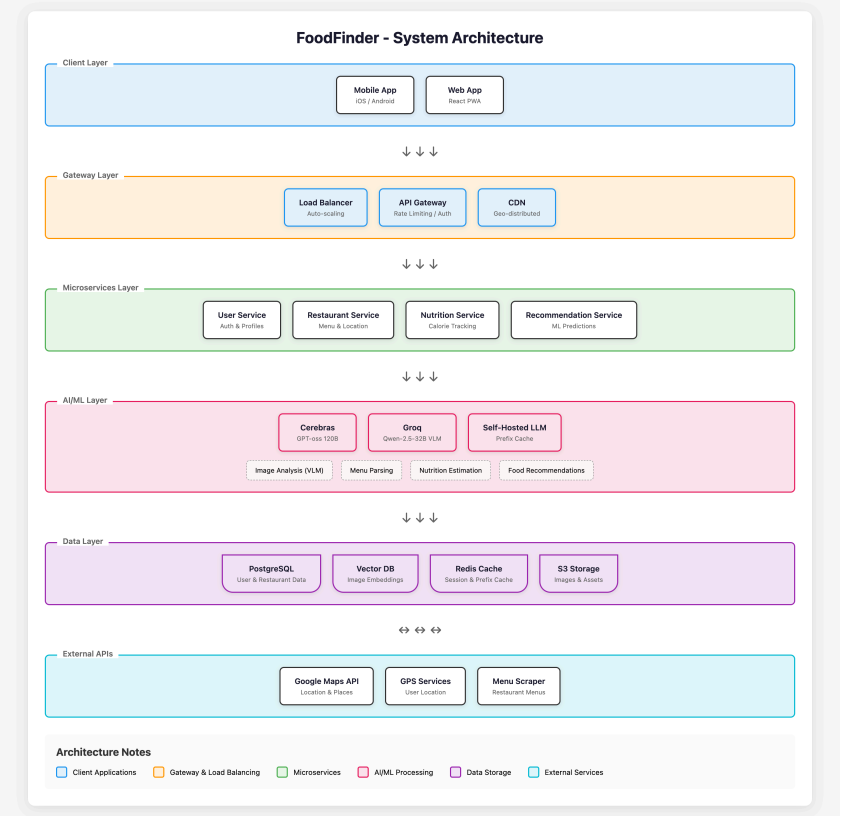
Feature Comparison and Competitive Advantages

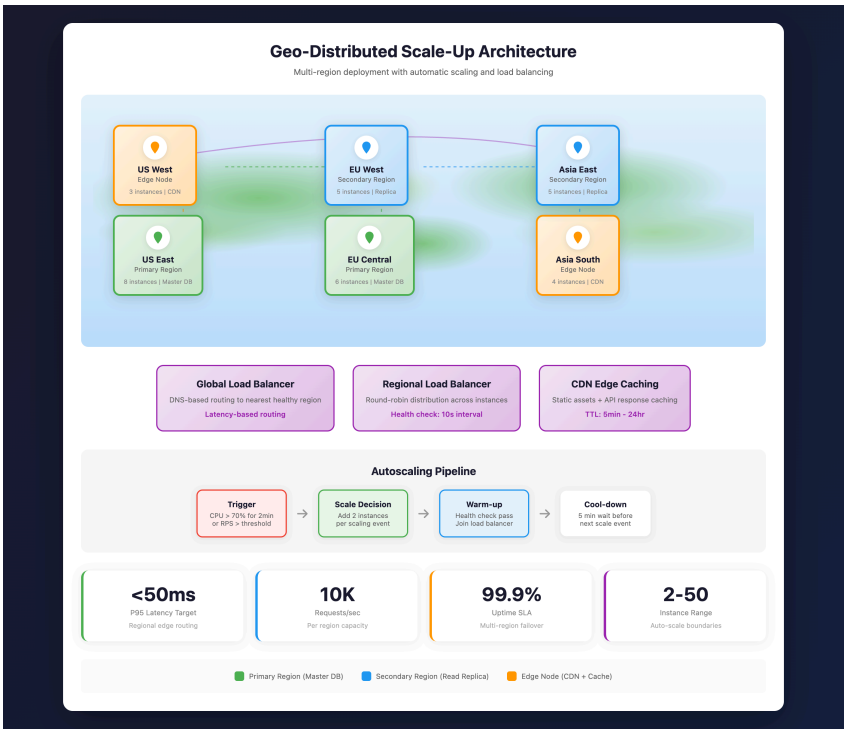
Feature	Google Maps	Yelp	Beli	GrubFindr
Recommendation level	Restaurant	Restaurant	Restaurant	**Dish-level**
Personalized taste learning	Basic (search/location history)	Minimal	Collaborative filtering	**Deep (dietary, budget, nutrition, context)**
Nutrition & dietary tracking	No	No	No	**Yes (calorie & macronutrient aware)**
AI/ML driven recommendations	Gemini (review summaries)	LLM (menu highlights)	Algorithmic/ML	**LLM + VLM (contextual dish selection)**
Food image analysis	Auto-labeling	Menu Vision (AR)	No	**VLM embedding & similarity search**
Decision output	List of restaurants	List of restaurants	Ranked restaurant list	**Single dish recommendation**
Social features	Shared lists	Reviews, photos	Strong (friend feed, taste matching)	Referral integrations
Onboarding friction	Low (Google account)	Low	High (must invite 4 friends)	**Low**
Primary demographic	All ages	Skews 55+	Gen Z (<35)	Health-conscious 18 - 45

GrubFindr differentiates on three axes: dish-level (not restaurant-level) recommendations, integrated nutrition and dietary awareness absent from all competitors, and LLM/VLM-powered context synthesis that delivers a single high-confidence recommendation rather than a list. None of the incumbents solve personalized, nutrition-aware dish selection.

System Architecture

GrubFindr follows a layered microservices architecture deployed across multiple geographic regions. The system is composed of six layers, each with distinct responsibilities. The full architecture and scale-up diagrams are shown below.





Client Layer: Users interact with GrubFindr through a native mobile app (iOS/Android) and a React-based progressive web app (PWA). Both clients communicate with the backend exclusively through the gateway layer.

Gateway Layer: A global load balancer routes traffic to the nearest healthy region using latency-based DNS routing. An API gateway handles authentication, rate limiting, and request validation. A geo-distributed CDN caches static assets and frequently accessed API responses with TTLs ranging from 5 minutes to 24 hours.

Microservices Layer: Four core services handle domain logic. The User Service manages authentication, profiles, and dietary preferences. The Restaurant Service manages restaurant metadata, menus, and geospatial location queries. The Nutrition Service tracks calorie and macronutrient data for menu items, sourcing data from VLM analysis and manual entry. The Recommendation Service orchestrates ML predictions to generate personalized dish-level suggestions based on user context.

AI/ML Layer: This layer powers the core intelligence of GrubFindr. Cerebras (Llama 3.3 70B) handles fast LLM inference for natural language queries and conversational recommendations. Groq (Llama 3.2 90B Vision) serves as the Vision Language Model for analyzing food images, extracting menu items from photos, and estimating nutritional content. A self-hosted LLM with per-user prefix caching maintains conversational context across sessions, enabling personalized recommendations that improve over time. An image analysis pipeline uses cosine similarity over 512-dimensional embeddings to match user-uploaded food photos against known menu items.

Data Layer: PostgreSQL serves as the primary relational database for users, restaurants, menu items, nutrition data, and meal logs, with PostGIS enabled for geospatial queries. A Vector Database (HNSW-indexed) stores 512-dimensional image embeddings for fast similarity search. Redis provides in-memory caching for user sessions, LLM prefix caches, API responses, and rate limiting counters. S3 object storage holds user-uploaded food images and restaurant menu photos with CDN integration.

External Services and Integrations: GrubFindr depends on the Google Maps API for location and place data, GPS services for real-time user positioning, and a menu scraper for ingesting restaurant menu data. GrubFindr provides services to external systems through referral and commission integrations: when a user decides on a dish, they can be redirected to partner platforms (e.g., Uber Eats, OpenTable) for ordering or reservations, with GrubFindr earning a commission on each conversion.

Scalability: The system is deployed across six regions (US West, US East, EU West, EU Central, Asia South, Asia East) with primary regions handling write operations and secondary regions serving read replicas. Autoscaling triggers when CPU exceeds 70% for 2 minutes or requests per second exceed the regional threshold, adding 2 instances per scaling event with a 5-minute cooldown. Target metrics include sub-50ms P95 latency, 10K requests/sec per region, and 99.9% uptime SLA.

Infrastructure

The following table lists all infrastructure components and third-party services required to build, deploy, and operate GrubFindr.

Component / Service	Role in System	Managed	Estimated Cost
AWS EC2 (compute)	Hosts microservices (User, Restaurant, Nutrition, Recommendation) across six regions	External (AWS)	~$2,400/mo (6 regions x 2 instances x m5.large on-demand); reducible to ~$1,500/mo with reserved instances
AWS Elastic Load Balancer	Global load balancing with latency-based DNS routing across regions	External (AWS)	~$120/mo (6 ALBs at ~$20/mo each)
Amazon CloudFront (CDN)	Caches static assets (PWA bundles, images) and frequently accessed API responses	External (AWS)	~$100/mo at moderate traffic; scales with usage
Amazon RDS for PostgreSQL + PostGIS	Primary relational database for users, restaurants, menu items, nutrition data, meal logs, and geospatial queries	External (AWS)	~$400/mo (db.r5.large primary + 2 read replicas in EU and Asia)
Vector Database (Pinecone or pgvector)	Stores 512-dimensional image embeddings with HNSW indexing for cosine similarity search	External (Pinecone) or Internal (pgvector extension)	~$70/mo (Pinecone Standard) or $0 additional if using pgvector on existing RDS
Amazon ElastiCache (Redis)	In-memory caching for user sessions, LLM prefix caches, API responses, and rate limiting counters	External (AWS)	~$150/mo (cache.r5.large with replication)
Amazon S3	Object storage for user-uploaded food images and restaurant menu photos	External (AWS)	~$25/mo at moderate storage volume
GPU Instances (AWS EC2 g5 or p3)	Self-hosted LLM and VLM inference for dish recommendations and food image analysis	Internal (self-hosted on AWS)	~$1,200/mo (2x g5.xlarge on-demand); reducible with spot instances
Google Maps Platform API	Location data, place search, geocoding, and distance calculations	External (Google)	$200 free credit/mo; ~$300/mo beyond free tier at moderate usage
Menu Scraper (custom)	Ingests and normalizes restaurant menu data from public sources	Internal	$0 (runs on existing compute); development cost only
React PWA + Mobile Apps	User-facing clients (iOS/Android native app, progressive web app)	Internal	$99/yr Apple Developer + $25 one-time Google Play = ~$125/yr

API Gateway (AWS API Gateway or Kong)	Authentication, rate limiting, request validation, and routing to microservices	External (AWS) or Internal (Kong)	~$50/mo (AWS API Gateway at moderate request volume)
Auth0 or AWS Cognito	User authentication, OAuth 2.0, social login, and session management	External	Free tier up to 7,500 MAU (Auth0) or 50,000 MAU (Cognito); ~$50/mo beyond
GitHub	Source code repository, version control, CI/CD via GitHub Actions	External (GitHub)	Free (Team plan: $4/user/mo if needed)
Docker + AWS ECR	Containerization and container image registry for microservice deployments	External (AWS)	~$10/mo (ECR storage)
AWS CloudWatch	Monitoring, logging, and alerting for all services and infrastructure	External (AWS)	~$50/mo at moderate log volume
Stripe	Payment processing for premium subscriptions	External (Stripe)	2.9% + $0.30 per transaction; no monthly fee
Domain + SSL (Route 53 + ACM)	DNS management and SSL/TLS certificates	External (AWS)	~$15/mo (Route 53 hosted zone + domain registration)

Total estimated monthly infrastructure cost at launch: approximately $4,900 - $5,100/mo, scaling with user growth. Significant cost reductions are achievable through reserved instances, spot GPU pricing, and leveraging free tiers during early growth.

Total Cost of Ownership (TCO)

The total cost of ownership for GrubFindr is estimated across four lifecycle stages: development, launch, operation, and growth/scaling. All figures assume a four-person development team and a 12-month initial timeline.

Stage 1: Development (Months 1 - 6)

Development costs cover initial system design, prototyping, and building the core platform. The primary cost is developer time. Assuming four team members working part-time (university project transitioning toward a startup), the imputed labor cost is approximately $0 in the academic phase, but would be ~$60,000 - $80,000 at market rates for a comparable six-month effort. Infrastructure costs during development are minimal: a single AWS region with small instances for testing (~$300/mo), Google Maps API within free tier, and GitHub free tier for source control and CI. Development tooling (IDEs, design tools) is free or covered by student licenses. Estimated total: ~$1,800 in cloud costs + $125 in app store fees = ~$2,000.

Stage 2: Launch (Months 6 - 8)

Launch costs include deploying to production across an initial two regions (US East and US West), provisioning production-grade databases, setting up monitoring, and marketing for initial user acquisition. Infrastructure scales to approximately $2,500/mo as production instances, managed databases, and GPU compute come online. Additional one-time costs include domain registration (~$15), app store listings ($125), and initial marketing spend (~$500 for social media and campus outreach). Estimated total for the launch phase: ~$5,600.

Stage 3: Steady-State Operation (Months 8 - 18)

Ongoing operational costs cover infrastructure, third-party API usage, and maintenance. At moderate scale (10,000 - 50,000 monthly active users), monthly costs stabilize around $4,900 - $5,100/mo as described in the Infrastructure section. The largest cost drivers are GPU compute for LLM/VLM inference (~$1,200/mo) and multi-region database hosting (~$400/mo). Stripe transaction fees begin contributing as premium subscriptions grow. An estimated $200/mo covers ongoing Google Maps API usage beyond the free tier. Estimated annual cost at steady state: ~$60,000 - $62,000.

Stage 4: Growth and Scaling (Months 18+)

As user volume grows beyond 50,000 MAU, costs scale primarily in compute and database layers. Additional regions (EU, Asia) add approximately $800 - $1,000/mo each. GPU costs grow with inference demand, potentially reaching $3,000 - $5,000/mo at scale, though model optimization and batching can offset this. Database read replicas, CDN bandwidth, and S3 storage scale roughly linearly with users. At 200,000 MAU, estimated monthly cost is $12,000 - $15,000/mo. Reserved instances and committed-use discounts become viable at this stage, reducing costs by 30 - 40%.

Lifecycle Stage	Duration	Estimated Cost	Primary Cost Drivers
Development	Months 1 - 6	~$2,000	Cloud testing environments, app store fees
Launch	Months 6 - 8	~$5,600	Production infrastructure, marketing, domain
Steady-State Operation	Months 8 - 18	~$60,000/yr	GPU inference, databases, API usage
Growth and Scaling	Months 18+	$12,000 - $15,000/mo at 200K MAU	Multi-region compute, GPU scaling, CDN bandwidth

Dependency and Vendor Risk Management

GrubFindr relies on several external dependencies. For each major dependency, the following identifies the vendor lock-in risk and proposes a migration or backup strategy.

Amazon Web Services (AWS)

Lock-in risk: High. GrubFindr uses multiple AWS-specific services (EC2, RDS, ElastiCache, S3, CloudFront, API Gateway, CloudWatch, ECR). Migrating the full stack to another cloud provider would require significant re-engineering of deployment configurations, monitoring, and managed service integrations.

Migration strategy: The microservices are containerized with Docker, making compute portable across any cloud or on-premise environment. PostgreSQL and Redis are open-source and available as managed services on all major clouds (GCP Cloud SQL, Azure Database for PostgreSQL). S3-compatible storage is available on GCP (Cloud Storage) and Azure (Blob Storage) with minimal API changes. To reduce lock-in further, infrastructure is defined as code using Terraform, enabling multi-cloud deployment with configuration changes rather than architectural rewrites. The most lock-in-prone services (CloudFront, API Gateway) can be replaced with vendor-neutral alternatives like Cloudflare and Kong.

Google Maps Platform

Lock-in risk: Moderate. GrubFindr uses Google Maps for geocoding, place search, and distance calculations. Google Maps is the most comprehensive and accurate mapping API, but Google controls pricing and could increase rates or restrict usage.

Migration strategy: The system abstracts location queries behind an internal service interface, allowing the underlying provider to be swapped. Viable alternatives include Mapbox (comparable coverage, often lower cost), OpenStreetMap/Nominatim (free, open-source geocoding), and HERE Maps. A phased migration could switch non-critical queries (e.g.,

static map tiles) to an alternative provider first while retaining Google for high-accuracy place search.

Self-Hosted LLM/VLM Models

Lock-in risk: Low to moderate. GrubFindr self-hosts its language and vision models on GPU instances, avoiding dependency on any single AI API provider. However, switching model architectures (e.g., from one open-source LLM to another) requires retraining, prompt re-engineering, and validation.

Migration strategy: The Recommendation Service uses a model-agnostic inference interface, so swapping the underlying model does not require changes to upstream services. Multiple open-source LLMs and VLMs are viable alternatives (e.g., LLaMA, Mistral, CLIP variants). If self-hosting becomes cost-prohibitive, the system can fall back to cloud AI APIs (OpenAI, Anthropic, Google Vertex AI) with minimal interface changes. Model weights and training data are stored internally, ensuring no vendor controls the core AI capability.

Stripe (Payment Processing)

Lock-in risk: Low. Stripe provides a well-documented, widely-used payment API, and the subscription billing integration is straightforward. However, migrating payment history and active subscriptions to a new provider requires careful handling.

Migration strategy: Alternatives include Braintree (PayPal), Square, and Adyen, all of which support similar subscription billing APIs. Customer payment methods would need to be re-tokenized during migration. Stripe's data export tools and API allow extraction of customer and subscription records. The payment integration layer is isolated behind an internal billing service, limiting the blast radius of a provider switch.

Auth0 / AWS Cognito (Authentication)

Lock-in risk: Moderate. User identity data, OAuth configurations, and social login integrations are tightly coupled to the auth provider. Migration requires careful handling of password hashes, session tokens, and third-party OAuth app credentials.

Migration strategy: Alternatives include Firebase Auth, Supabase Auth, or a self-hosted solution using Keycloak. Both Auth0 and Cognito support user data export. The system stores user profile data in its own PostgreSQL database rather than relying solely on the auth provider's user store, reducing the scope of migration. OAuth social login credentials (Google, Apple) are owned by the GrubFindr developer accounts and are provider-independent.

Knowledge Sharing and Project Continuity

GrubFindr ensures project continuity through documentation standards, knowledge transfer practices, and shared tooling so that the departure of any single team member does not stall development.

Documentation and Code Standards

All code follows a consistent style enforced by linters (ESLint for TypeScript/React, Black for Python microservices). Each microservice repository includes a README with setup instructions, environment variable documentation, and API endpoint descriptions. Architecture Decision Records (ADRs) document key technical decisions with context, alternatives considered, and rationale. The data model and API contracts are documented using OpenAPI (Swagger) specifications, which serve as the single source of truth for inter-service communication and are kept in version control alongside the code.

Knowledge Transfer Practices

The team holds weekly meetings where each member presents their current work and any blockers. All meeting minutes are recorded in the project diary (Appendix A). Code changes are submitted via pull requests on GitHub, requiring at least one approving review from another team member before merging. This ensures that no piece of code is understood by only one person. Onboarding documentation for new contributors is maintained in the repository wiki, covering local development setup, deployment procedures, and the overall system architecture. Each team member is designated a secondary owner of another member's

primary domain (e.g., Andre is primary on architecture, with Jeffrey as secondary) to ensure redundancy.

Tooling

GitHub serves as the central hub for source control, issue tracking, project boards, and CI/CD via GitHub Actions. All infrastructure is defined as code using Terraform, stored in a dedicated repository, so any team member can review, modify, or redeploy infrastructure. Docker Compose files enable any developer to run the full system locally. Shared secrets and environment variables are managed through AWS Secrets Manager with role-based access, rather than being stored on individual machines. The CI/CD pipeline automates testing, building, and deploying to staging and production environments, reducing reliance on any single person's knowledge of the deployment process.

Implementation Status

Feature	Status	Notes
System architecture design	Implemented	Layered microservices architecture with full diagrams
Database schema design	Implemented	ER diagram, replication strategy, and index plan finalized
UI/UX mock-ups	Implemented	Mobile and web interactive prototypes created
Competitive analysis and market research	Implemented	Google Maps, Yelp, Beli analyzed with feature comparison
Infrastructure and cost planning	Implemented	Full TCO across four lifecycle stages
Frontend (React PWA)	In Progress (20%)	Component scaffolding started; mock-ups guide implementation
User Service (auth, profiles, preferences)	In Progress (15%)	API contracts defined; integration with Auth0/Cognito planned
Restaurant Service (menus, geospatial queries)	In Progress (10%)	Schema finalized; PostGIS queries prototyped
Menu scraper	In Progress (10%)	Target sources identified; scraper logic in early development
Nutrition Service (calorie/macro tracking)	To Do	Dependent on VLM integration and menu data pipeline
Recommendation Service (LLM dish selection)	To Do	Cerebras and Groq API integration planned for Phase 3
VLM image analysis pipeline	To Do	Embedding generation and similarity search via vector DB
Native mobile app (iOS/Android)	To Do	Will share logic with PWA; scheduled for Phase 3
CI/CD pipeline (GitHub Actions)	To Do	Automated testing, build, and deployment to AWS
Multi-region deployment	To Do	Terraform IaC and autoscaling configuration

Payment integration (Stripe)	To Do	Premium subscription billing for paid tier

Conclusion

GrubFindr addresses the growing problem of meal decision fatigue by consolidating fragmented restaurant and dish data into a single, personalized recommendation engine powered by LLMs and VLMs. The platform monetizes through a freemium subscription model, sponsored dish placements, and referral commissions. By focusing on dish-level personalization rather than generic restaurant discovery, GrubFindr differentiates itself from incumbents like Google Maps, Yelp, and Beli, positioning itself as the decision-first entry point in the meal discovery process. The platform is well-suited to capture high-intent demand in a market increasingly driven by AI-powered personalization. Architecturally, the system is built on a layered microservices design with dedicated AI/ML inference providers (Cerebras for LLM, Groq for VLM), a polyglot data layer spanning PostgreSQL, Redis, a Vector Database, and S3, and a geo-distributed multi-region deployment with autoscaling to meet sub-50ms latency and 99.9% uptime targets at scale.

Appendix A: Project Diary

February 2nd Team Meeting

Attendees: Abyan Haq, Andre Slavescu, Bilal Nasar, Jeffrey Qingjie Xuzhang
Discussion:

- Discussed project scope and brainstormed app concepts
- Decided on GrubFindr as the project direction
- Assigned initial research areas

Contributions:

- Bilal: Created group chat, set up shared Google Drive, drafted project timeline
- Andre: Researched existing food recommendation APIs and LLM integration feasibility
- Jeffrey: Conducted initial competitor analysis (Google Maps, Yelp, Beli)
- Abyan: Narrowed down problem scope and crafted a story explaining why GrubFindr

To-Dos (due February 5th):

- Bilal: Research findings and define business model structure
- Andre: Create system architecture diagram and ER model
- Abyan: Start working on slides and set up the problem statements
- Jeffrey: Conduct market analysis and find differentiation from competitors

To-Dos for Later Phases (due February 20th):

- Bilal: Dive deeper into system costs and how costs will scale with more usage
- Andre: Build mock prototypes with digital tools such as Figma / Claude
- Abyan: Work on creating a live product demo to show others how to use the app
- Jeffrey: Start working on the 5-page TCO report and structure with initial findings and research